

LAPORAN PRAKTIKUM
POSTTEST(7)
ALGORITMA PEMROGRAMAN LANJUT



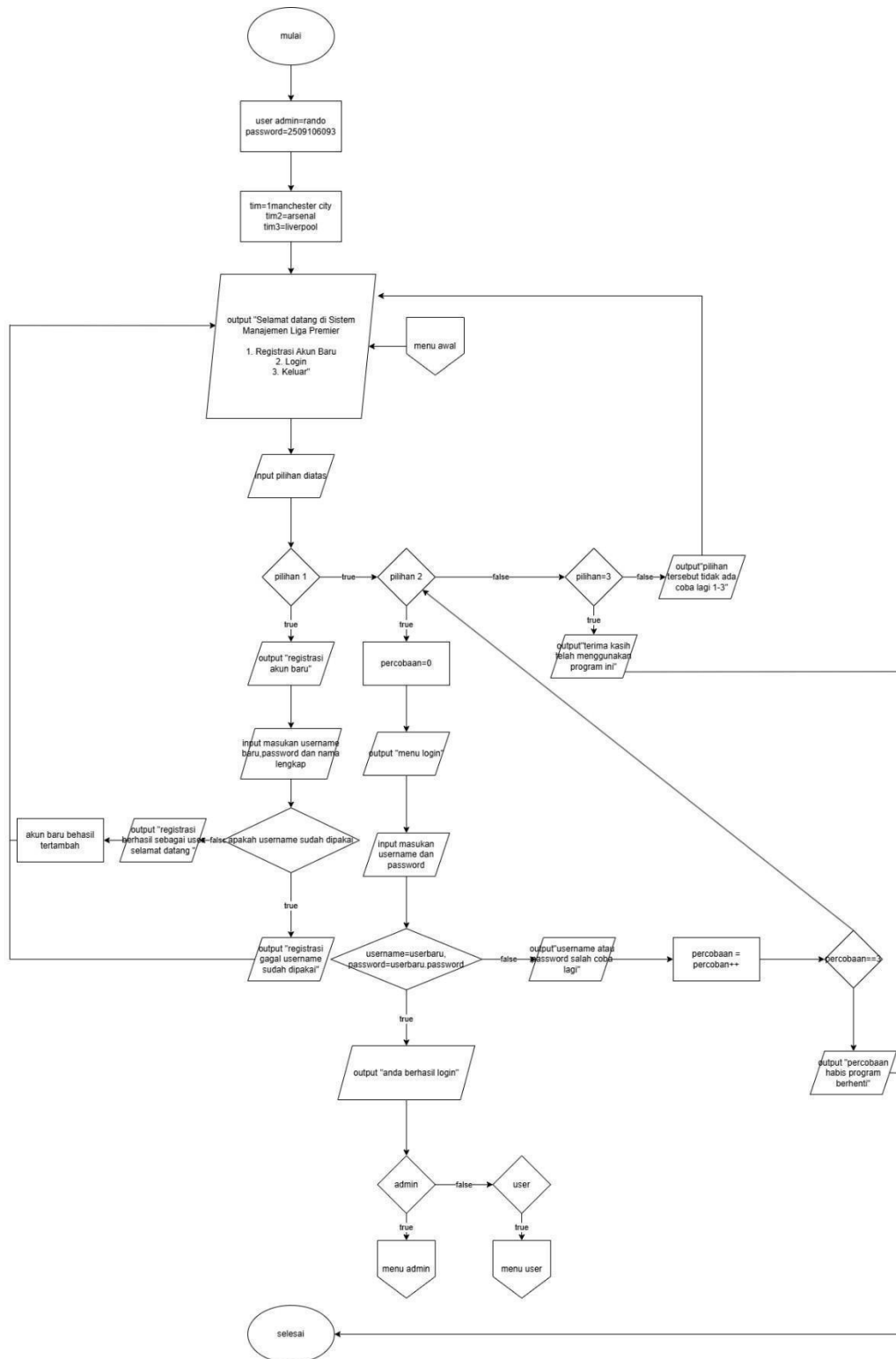
Disusun oleh:

Nama: Isrando Mirhjah (2509106093)

Kelas (C"25)

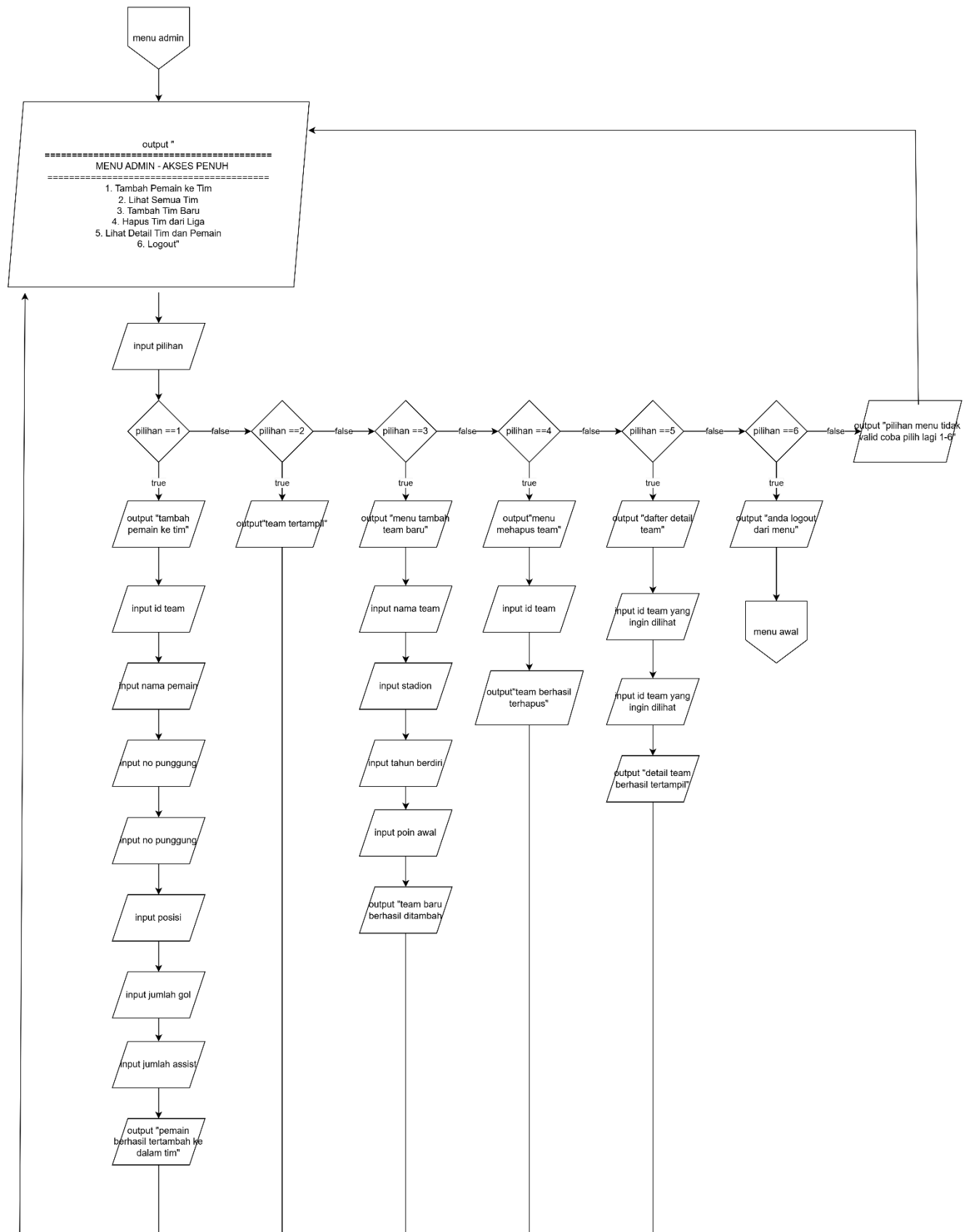
PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2025

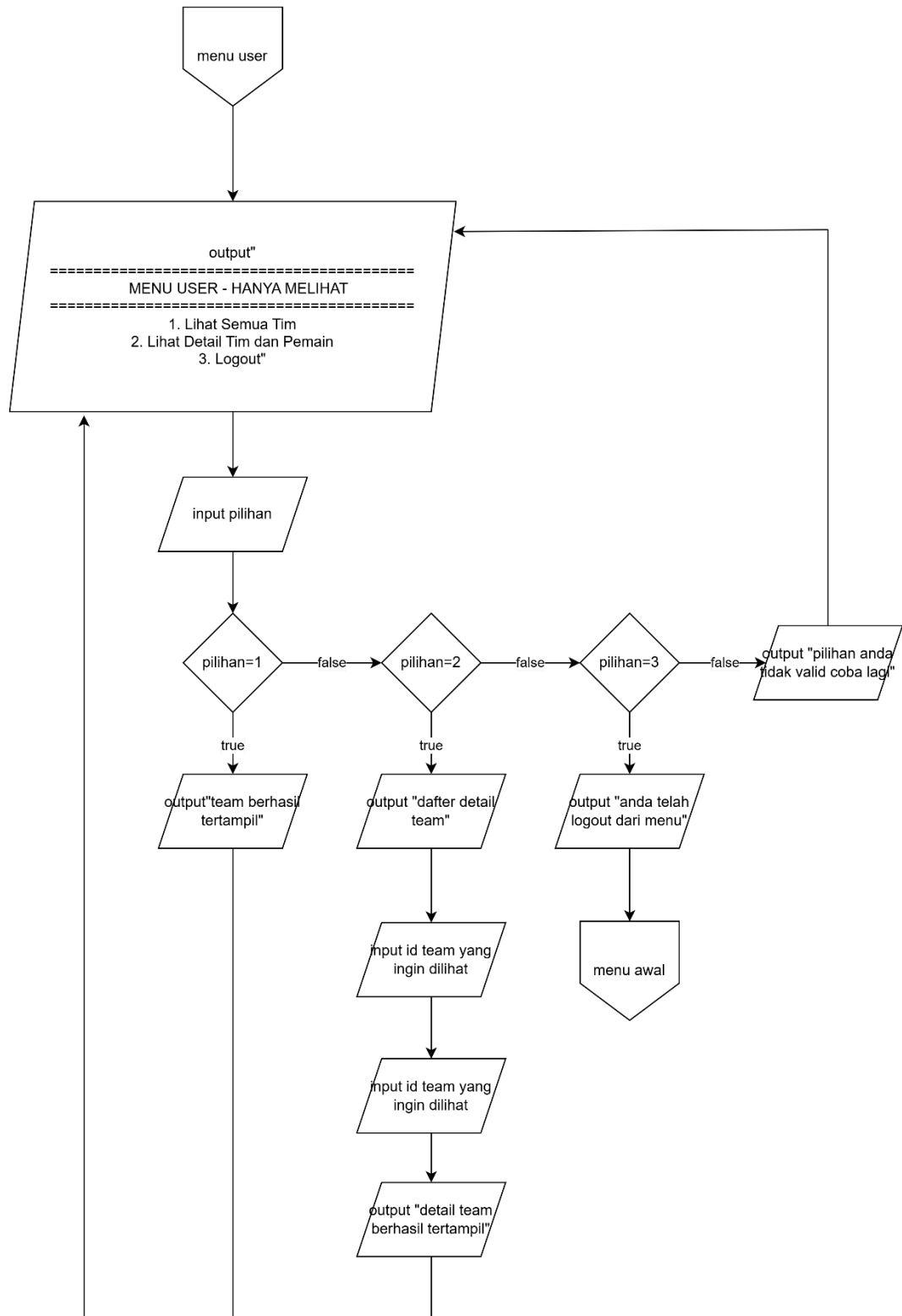
1. Flowchart



Flowchart 1.1 main menu

Flowchart 1.2 menu admin





Flowchart 1.3 menu user

Flowchart dia atas untuk menggambarkan cara alur program yang telah dibuat

2. Deskripsi Singkat Program

Program ini dibuat untuk agar kita bisa melihat detail team yang ada di premier league dan kita bisa menambahkan team baru dan menghapus team lama,kita juga bisa menambahkan pemain ke dalam team

3. Source Code

```

#include <iostream>
#include <iomanip>
#include <string>
#include <algorithm>

using namespace std;

const size_t MAX_STREAM_SIZE = 1000;
const int INT_MIN_VALUE = -2147483648;
const int INT_MAX_VALUE = 2147483647;

void clearInputBuffer() {
    cin.ignore(MAX_STREAM_SIZE, '\n');
}

void clearScreenManual() {
    for (int i = 0; i < 50; i++) {
        cout << endl;
    }
}

class InputException : public exception {
public:
    const char* what() const noexcept override {
        return "Input tidak valid!";
    }
};

class RangeException : public exception {
    string message;
public:
    RangeException(int minVal, int maxVal) {
        message = "Input harus antara " + to_string(minVal) + " - " +
to_string(maxVal) + "!";
    }
    const char* what() const noexcept override {
        return message.c_str();
    }
};

class DatabaseException : public exception {
    string message;
public:
    DatabaseException(const string& msg) : message(msg) {}
    const char* what() const noexcept override {
        return message.c_str();
    }
};

int Integer(string pesan, int minValue, int maxValue) {
    while (true) {
        try {
            cout << pesan;
            int angka;
            cin >> angka;

```

```

        if (cin.fail()) {
            cin.clear();
            clearInputBuffer();
            throw InputException();
        }

        if (angka < minValue || angka > maxValue) {
            clearInputBuffer();
            throw RangeException(minValue, maxValue);
        }

        clearInputBuffer();
        return angka;
    }
    catch (const InputException& e) {
        cout << e.what() << " Input harus berupa angka! Silakan coba
lagi.\n";
    }
    catch (const RangeException& e) {
        cout << e.what() << " Silakan coba lagi.\n";
    }
    catch (...) {
        cout << "Terjadi kesalahan tidak terduga. Silakan coba
lagi.\n";
        clearInputBuffer();
    }
}

int Integer(string pesan) {
    return Integer(pesan, INT_MIN_VALUE, INT_MAX_VALUE);
}

struct Pemain {
    string namaPemain;
    int nomorPunggung;
    string posisi;
    int gol;
    int assist;
};

struct Tim {
    int id;
    string namaTim;
    string stadion;
    int tahunBerdiri;
    int poin;
    Pemain* skuad;
    int jumlahPemain;
    int kapasitasPemain;
};

struct User {
    string username;

```



```

    string password;
    string role;
    string namaLengkap;
};

struct UserNode {
    User data;
    UserNode* next;
};

class UserDatabase {
private:
    UserNode* head;
    int size;

public:
    UserDatabase() : head(nullptr), size(0) {}

    ~UserDatabase() {
        UserNode* current = head;
        while (current != nullptr) {
            UserNode* temp = current;
            current = current->next;
            delete temp;
        }
    }

    bool addUser(User user) {
        if (findUser(user.username) != nullptr) {
            throw DatabaseException("Username sudah digunakan!");
        }
        UserNode* newNode = new UserNode;
        newNode->data = user;
        newNode->next = head;
        head = newNode;
        size++;
        return true;
    }

    User* findUser(string username) {
        UserNode* current = head;
        while (current != nullptr) {
            if (current->data.username == username) {
                return &(current->data);
            }
            current = current->next;
        }
        return nullptr;
    }
};

const int MAKS_TIM = 20;
UserDatabase databaseUsers;

void inisialisasiTim(Tim* tim, int id, string nama, string stadion, int

```

```

tahun, int poin) {
    tim->id = id;
    tim->namaTim = nama;
    tim->stadion = stadion;
    tim->tahunBerdiri = tahun;
    tim->poin = poin;
    tim->jumlahPemain = 0;
    tim->kapasitasPemain = 11;
    tim->skواد = new Pemain[tim->kapasitasPemain];
}

void tambahPemainKeTim(Tim* tim, Pemain* pemainBaru, bool* berhasil) {
    if (tim->jumlahPemain < tim->kapasitasPemain) {
        tim->skواد[tim->jumlahPemain] = *pemainBaru;
        tim->jumlahPemain++;
        *berhasil = true;
    } else {
        *berhasil = false;
    }
}

Pemain* cariPemainByNomor(Tim* tim, int nomorPunggung) {
    for (int i = 0; i < tim->jumlahPemain; i++) {
        if (tim->skواد[i].nomorPunggung == nomorPunggung) {
            return &(tim->skواد[i]);
        }
    }
    return nullptr;
}

int hitungTotalGol(Pemain* skواد, int indeks, int jumlahPemain) {
    if (indeks >= jumlahPemain) return 0;
    return skواد[indeks].gol + hitungTotalGol(skواد, indeks + 1,
jumlahPemain);
}

int hitungTotalAssist(Pemain* skواد, int indeks, int jumlahPemain) {
    if (indeks >= jumlahPemain) return 0;
    return skواد[indeks].assist + hitungTotalAssist(skواد, indeks + 1,
jumlahPemain);
}

void tampilkanPemain(Pemain* skواد, int indeks, int jumlahPemain) {
    if (indeks >= jumlahPemain) return;
    cout << left
        << setw(5) << indeks + 1
        << setw(25) << skواد[indeks].namaPemain.substr(0, 24)
        << setw(15) << skواد[indeks].nomorPunggung
        << setw(15) << skواد[indeks].posisi.substr(0, 14)
        << setw(10) << skواد[indeks].gol
        << setw(10) << skواد[indeks].assist
        << "\n";
    tampilkanPemain(skواد, indeks + 1, jumlahPemain);
}

```

```

void clearScreen() {
    clearScreenManual();
}

void tampilkanHeader(string judul) {
    cout << "\n=====\\n";
    cout << "    " << judul << "\\n";
    cout << "=====\\n";
}

void tampilkanGaris(char ch = '=', int panjang = 50) {
    cout << string(panjang, ch) << endl;
}

void tampilkanPesan(string pesan, bool tungguEnter = true) {
    cout << pesan << endl;
    if (tungguEnter) {
        cout << "Tekan Enter untuk melanjutkan...";
        cin.get();
    }
}

int cariTim(int idTim, int jumlahTim, Tim daftarTim[]) {
    for (int i = 0; i < jumlahTim; i++) {
        if (daftarTim[i].id == idTim) return i;
    }
    return -1;
}

string toLower(string str) {
    string hasil = str;
    transform(hasil.begin(), hasil.end(), hasil.begin(), ::tolower);
    return hasil;
}

int sequentialSearchTimByNama(Tim* daftarTim, int* jumlahTim, string*
keyword, int* hasilIndeks) {
    string keyLower = toLower(*keyword);
    int jumlahHasil = 0;
    for (int i = 0; i < *jumlahTim; i++) {
        if (toLower(daftarTim[i].namaTim).find(keyLower) !=
string::npos) {
            hasilIndeks[jumlahHasil] = i;
            jumlahHasil++;
        }
    }
    return jumlahHasil;
}

void menuCariTimById(Tim* daftarTim, int* jumlahTim, int* nextIdTim) {
    clearScreen();
    tampilkanHeader("CARI TIM BERDASARKAN ID");
    tampilkanGaris('-', 50);

    if (*jumlahTim == 0) {

```

```

        tampilkanPesan("Belum ada tim terdaftar.", true);
        return;
    }

    cout << "\nID Tim yang tersedia: ";
    for (int i = 0; i < *jumlahTim; i++) {
        cout << daftarTim[i].id;
        if (i < *jumlahTim - 1) cout << ", ";
    }
    cout << "\n\n";

    try {
        int targetId = Integer("Masukkan ID Tim yang dicari: ", 1,
INT_MAX_VALUE);
        cout << "\nMencari tim dengan ID = " << targetId << " ...\n";
        tampilkanGaris('-', 50);

        int indeks = -1;
        for (int i = 0; i < *jumlahTim; i++) {
            if (daftarTim[i].id == targetId) {
                indeks = i;
                break;
            }
        }

        if (indeks == -1) {
            cout << "\n[TIDAK DITEMUKAN] Tim dengan ID " << targetId <<
" tidak ada.\n";
        } else {
            cout << "\n[DITEMUKAN] Tim ditemukan!\n\n";
            cout << left << setw(5) << "ID" << setw(25) << "Nama Tim" <<
setw(25) << "Stadion"
                << setw(15) << "Tahun" << setw(10) << "Poin" <<
setw(15) << "Pemain\n";
            tampilkanGaris('-', 95);
            cout << left << setw(5) << daftarTim[indeks].id
                << setw(25) << daftarTim[indeks].namaTim.substr(0, 24)
                << setw(25) << daftarTim[indeks].stadion.substr(0, 24)
                << setw(15) << daftarTim[indeks].tahunBerdiri
                << setw(10) << daftarTim[indeks].poin
                << setw(15) << daftarTim[indeks].jumlahPemain << "\n";
            tampilkanGaris('=', 95);
        }
    } catch (...) {
        cout << "Terjadi kesalahan saat mencari tim.\n";
    }

    cout << "\nTekan Enter untuk kembali...";
    cin.get();
}

void menuCariTimByNama(Tim* daftarTim, int* jumlahTim) {
    clearScreen();
    tampilkanHeader("CARI TIM BERDASARKAN NAMA");
    tampilkanGaris('-', 50);

```

```

    if (*jumlahTim == 0) {
        tampilkanPesan("Belum ada tim terdaftar.", true);
        return;
    }

    try {
        cout << "Masukkan nama tim (atau sebagian nama): ";
        string keyword;
        getline(cin, keyword);

        int hasilIndeks[MAKS_TIM];
        int jumlahHasil = sequentialSearchTimByNama(daftarTim,
            jumlahTim, &keyword, hasilIndeks);

        cout << "\nMencari dengan kata kunci \"" << keyword << "\"
... \n";
        tampilkanGaris('-', 95);

        if (jumlahHasil == 0) {
            cout << "\n[TIDAK DITEMUKAN] Tidak ada tim yang sesuai.\n";
        } else {
            cout << "\n[DITEMUKAN] " << jumlahHasil << " tim
ditemukan:\n\n";
            cout << left << setw(5) << "ID" << setw(25) << "Nama Tim" <<
setw(25) << "Stadion"
                << setw(15) << "Tahun" << setw(10) << "Poin" <<
setw(15) << "Pemain\n";
            tampilkanGaris('-', 95);

            for (int i = 0; i < jumlahHasil; i++) {
                int idx = hasilIndeks[i];
                cout << left << setw(5) << daftarTim[idx].id
                    << setw(25) << daftarTim[idx].namaTim.substr(0, 24)
                    << setw(25) << daftarTim[idx].stadion.substr(0, 24)
                    << setw(15) << daftarTim[idx].tahunBerdiri
                    << setw(10) << daftarTim[idx].poin
                    << setw(15) << daftarTim[idx].jumlahPemain << "\n";
            }
            tampilkanGaris('=', 95);
        }
    } catch (...) {
        cout << "Terjadi kesalahan saat mencari tim.\n";
    }

    cout << "\nTekan Enter untuk kembali...";
    cin.get();
}

bool validasiTahunBerdiri(int tahun) {
    return (tahun >= 1800 && tahun <= 2024);
}

bool validasiNomorPunggung(int nomor, Tim tim) {
    if (nomor < 1 || nomor > 99) return false;
}

```

```

        for (int i = 0; i < tim.jumlahPemain; i++) {
            if (tim.skvad[i].nomorPunggung == nomor) return false;
        }
        return true;
    }

    bool registrasi(User* userBaru) {
        if (databaseUsers.findUser(userBaru->username) != nullptr) {
            throw DatabaseException("Username sudah digunakan!");
        }
        if (userBaru->password.length() < 6) {
            throw DatabaseException("Password harus minimal 6 karakter!");
        }
        userBaru->role = "User";
        return databaseUsers.addUser(*userBaru);
    }

    bool loginUser(string username, string password, User* currentUser) {
        User* user = databaseUsers.findUser(username);
        if (user != nullptr && user->password == password) {
            *currentUser = *user;
            return true;
        }
        return false;
    }

    bool tambahTimBaru(Tim* daftarTim, int* jumlahTim, int* nextId, Tim*
timBaru) {
        if (*jumlahTim >= MAKS_TIM) {
            throw DatabaseException("Database tim sudah penuh!");
        }
        timBaru->id = (*nextId)++;
        timBaru->jumlahPemain = 0;
        timBaru->kapasitasPemain = 11;
        timBaru->skvad = new Pemain[timBaru->kapasitasPemain];
        daftarTim[*jumlahTim] = *timBaru;
        (*jumlahTim)++;
        return true;
    }

    bool hapusTimDariLiga(Tim* daftarTim, int* jumlahTim, int idTim) {
        int indeks = cariTim(idTim, *jumlahTim, daftarTim);
        if (indeks == -1) return false;

        delete[] daftarTim[indeks].skvad;
        daftarTim[indeks].skvad = nullptr;

        for (int i = indeks; i < *jumlahTim - 1; i++) {
            daftarTim[i] = daftarTim[i + 1];
        }
        daftarTim[*jumlahTim - 1].skvad = nullptr;
        (*jumlahTim)--;
        return true;
    }
}

```

```

void registrasiMenu() {
    clearScreen();
    tampilkanHeader("REGISTRASI AKUN BARU");

    try {
        User userBaru;

        cout << "Masukkan Username: ";
        getline(cin, userBaru.username);
        cout << "Masukkan Password (minimal 6 karakter): ";
        getline(cin, userBaru.password);
        cout << "Masukkan Nama Lengkap: ";
        getline(cin, userBaru.namaLengkap);

        if (registrasi(&userBaru)) {
            cout << "\n=====\\n";
            cout << "                REGISTRASI BERHASIL!\\n";
            cout << "=====\\n";
            cout << "Username      : " << userBaru.username << "\\n";
            cout << "Nama Lengkap: " << userBaru.namaLengkap << "\\n";
            cout << "Role          : User\\n";
            cout << "=====\\n";
            cout << "Silakan login menggunakan akun Anda.\\n";
        }
    }
    catch (const DatabaseException& e) {
        cout << "\n=====\\n";
        cout << "                REGISTRASI GAGAL!\\n";
        cout << "=====\\n";
        cout << e.what() << "\\n";
        cout << "=====\\n";
    }
    catch (...) {
        cout << "\\nTerjadi kesalahan tidak terduga saat registrasi.\\n";
    }

    cout << "\\nTekan Enter untuk melanjutkan...";
    cin.get();
}

void loginMenu(User* currentUser) {
    string username, password;
    int percobaan = 0;
    const int maxPercobaan = 3;

    while (percobaan < maxPercobaan) {
        clearScreen();
        tampilkanHeader("MENU LOGIN");
        cout << "Username: ";
        getline(cin, username);
        cout << "Password: ";
        getline(cin, password);

        if (loginUser(username, password, currentUser)) {
            tampilkanGaris('-');
        }
    }
}

```

```

        cout << "\nLOGIN BERHASIL!\n";
        cout << "Selamat datang, " << currentUser->namaLengkap <<
"!\\n";

        cout << "Role: " << currentUser->role << "\\n";
        tampilkanGaris('-');
        cout << "\\nTekan Enter untuk melanjutkan...";
        cin.get();
        return;
    }

    percobaan++;
    if (percobaan < maxPercobaan) {
        cout << "\\nUsername atau password salah!\n";
        cout << "Sisa percobaan: " << (maxPercobaan - percobaan) <<
"\\n";

        cout << "Tekan Enter untuk melanjutkan...";
        cin.get();
    }
}

clearScreen();
tampilkanHeader("LOGIN GAGAL");
cout << "\\nAnda telah gagal login sebanyak " << maxPercobaan << "
kali.\\n";
cout << "Tekan Enter untuk kembali ke menu utama...";
cin.get();
}

void lihatSemuaTim(Tim* daftarTim, int* jumlahTim) {
    clearScreen();
    tampilkanHeader("DAFTAR SEMUA TIM");

    if (*jumlahTim == 0) {
        tampilkanPesan("Belum ada tim terdaftar.", true);
        return;
    }

    cout << left << setw(5) << "ID" << setw(25) << "Nama Tim" <<
setw(25) << "Stadion"
        << setw(15) << "Tahun" << setw(10) << "Poin" << setw(15) <<
"Pemain\\n";
    tampilkanGaris('-', 95);

    for (int i = 0; i < *jumlahTim; i++) {
        cout << left << setw(5) << daftarTim[i].id
            << setw(25) << daftarTim[i].namaTim.substr(0, 24)
            << setw(25) << daftarTim[i].stadion.substr(0, 24)
            << setw(15) << daftarTim[i].tahunBerdiri
            << setw(10) << daftarTim[i].poin
            << setw(15) << daftarTim[i].jumlahPemain << "\\n";
    }
    tampilkanGaris('=', 95);

    cout << "\\nTekan Enter untuk kembali...";
    cin.get();
}

```



```

}

void lihatDetailTim(Tim* daftarTim, int* jumlahTim, int* nextIdTim) {
    clearScreen();
    tampilkanHeader("DETAIL TIM DAN PEMAIN");

    if (*jumlahTim == 0) {
        tampilkanPesan("Belum ada tim terdaftar.", true);
        return;
    }

    lihatSemuaTim(daftarTim, jumlahTim);

    try {
        int idTim = Integer("\nMasukkan ID Tim: ", 1, *nextIdTim - 1);
        int indeks = cariTim(idTim, *jumlahTim, daftarTim);

        if (indeks == -1) {
            tampilkanPesan("Tim tidak ditemukan!", true);
            return;
        }

        clearScreen();
        cout << "\n=== DETAIL TIM ===\n";
        cout << "ID Tim          : " << daftarTim[indeks].id << "\n";
        cout << "Nama Tim           : " << daftarTim[indeks].namaTim <<
"\n";
        cout << "Stadion            : " << daftarTim[indeks].stadion <<
"\n";
        cout << "Tahun Berdiri     : " << daftarTim[indeks].tahunBerdiri <<
"\n";
        cout << "Poin               : " << daftarTim[indeks].poin << "\n";
        cout << "Jumlah Pemain     : " << daftarTim[indeks].jumlahPemain <<
"/"
        << daftarTim[indeks].kapasitasPemain << "\n";

        if (daftarTim[indeks].jumlahPemain > 0) {
            cout << "\nDAFTAR PEMAIN:\n";
            tampilkanGaris('-', 80);
            cout << left << setw(5) << "No" << setw(25) << "Nama Pemain"
<< setw(15) << "No Punggung"
            << setw(15) << "Posisi" << setw(10) << "Gol" <<
setw(10) << "Assist\n";
            tampilkanGaris('-', 80);
            tampilkanPemain(daftarTim[indeks].skواد, 0,
daftarTim[indeks].jumlahPemain);
            tampilkanGaris('=', 80);

            cout << "\nTotal Gol   : " <<
hitungTotalGol(daftarTim[indeks].skواد, 0,
daftarTim[indeks].jumlahPemain) << "\n";
            cout << "Total Assist: " <<
hitungTotalAssist(daftarTim[indeks].skواد, 0,
daftarTim[indeks].jumlahPemain) << "\n";
        }
    }
}

```

```

    } catch (...) {
        cout << "Terjadi kesalahan saat menampilkan detail tim.\n";
    }

    cout << "\nTekan Enter untuk kembali...";
    cin.get();
}

void tambahTimMenu(Tim* daftarTim, int* jumlahTim, int* nextIdTim) {
    clearScreen();
    tampilkanHeader("TAMBAH TIM BARU");

    if (*jumlahTim >= MAKS_TIM) {
        tampilkanPesan("Database tim sudah penuh!", true);
        return;
    }

    try {
        Tim timBaru;
        cout << "Nama Tim: ";
        getline(cin, timBaru.namaTim);
        cout << "Stadion: ";
        getline(cin, timBaru.stadion);
        timBaru.tahunBerdiri = Integer("Tahun Berdiri (1800-2024): ",
1800, 2024);
        timBaru.poin = Integer("Poin Awal (0-100): ", 0, 100);

        if (tambahTimBaru(daftarTim, jumlahTim, nextIdTim, &timBaru)) {
            tampilkanPesan("\nTim berhasil ditambahkan dengan ID: " +
to_string(daftarTim[*jumlahTim-1].id), true);
        }
    } catch (const DatabaseException& e) {
        cout << "Gagal: " << e.what() << "\n";
    } catch (...) {
        cout << "Terjadi kesalahan saat menambahkan tim.\n";
    }
}

void tambahPemainMenu(Tim* daftarTim, int* jumlahTim, int* nextIdTim) {
    clearScreen();
    tampilkanHeader("TAMBAH PEMAIN KE TIM");

    if (*jumlahTim == 0) {
        tampilkanPesan("Belum ada tim. Tambahkan tim terlebih dahulu.",
true);
        return;
    }

    cout << "DAFTAR TIM:\n";
    tampilkanGaris('-', 50);
    for (int i = 0; i < *jumlahTim; i++) {
        cout << "ID: " << daftarTim[i].id << " | " <<
daftarTim[i].namaTim << "\n";
    }
    tampilkanGaris('-', 50);
}

```

```

try {
    int idTim = Integer("\nMasukkan ID Tim: ", 1, *nextIdTim - 1);
    int idx = cariTim(idTim, *jumlahTim, daftarTim);

    if (idx == -1) {
        tampilkanPesan("Tim tidak ditemukan!", true);
        return;
    }

    if (daftarTim[idx].jumlahPemain >=
daftarTim[idx].kapasitasPemain) {
        tampilkanPesan("Tim sudah penuh!", true);
        return;
    }

    Pemain p;
    cout << "\nNama Pemain: ";
    getline(cin, p.namaPemain);

    do {
        p.nomorPunggung = Integer("Nomor Punggung (1-99): ", 1, 99);
    } while (!validasiNomorPunggung(p.nomorPunggung,
daftarTim[idx]));

    cout << "Posisi: ";
    getline(cin, p.posisi);
    p.gol = Integer("Gol: ", 0, 999);
    p.assist = Integer("Assist: ", 0, 999);

    bool berhasil = false;
    tambahPemainKeTim(&daftarTim[idx], &p, &berhasil);

    if (berhasil) {
        tampilkanPesan("\nPemain berhasil ditambahkan!", true);
    }
} catch (...) {
    cout << "Terjadi kesalahan saat menambahkan pemain.\n";
}
}

void hapusTimMenu(Tim* daftarTim, int* jumlahTim, int* nextIdTim) {
    clearScreen();
    tampilkanHeader("HAPUS TIM DARI LIGA");

    if (*jumlahTim == 0) {
        tampilkanPesan("Belum ada tim.", true);
        return;
    }

    lihatSemuaTim(daftarTim, jumlahTim);

    try {
        int idTim = Integer("\nMasukkan ID Tim yang akan dihapus: ", 1,
*nextIdTim - 1);

```

```

        char konfirmasi;

        cout << "\nYakin ingin menghapus? (y/n): ";
        cin >> konfirmasi;
        clearInputBuffer();

        if (konfirmasi == 'y' || konfirmasi == 'Y') {
            if (hapusTimDariLiga(daftarTim, jumlahTim, idTim)) {
                tampilkanPesan("\nTim berhasil dihapus!", true);
            } else {
                tampilkanPesan("\nGagal menghapus tim!", true);
            }
        } else {
            tampilkanPesan("\nPenghapusan dibatalkan.", true);
        }
    } catch (...) {
        cout << "Terjadi kesalahan saat menghapus tim.\n";
    }
}

void menuSearching(Tim* daftarTim, int* jumlahTim, int* nextIdTim) {
    int pilihan;
    bool running = true;
    do {
        clearScreen();
        tampilkanHeader("MENU PENCARIAN TIM");
        cout << "1. Cari berdasarkan ID\n";
        cout << "2. Cari berdasarkan Nama\n";
        cout << "3. Kembali\n\n";
        pilihan = Integer("Pilih (1-3): ", 1, 3);

        if (pilihan == 1) menuCariTimById(daftarTim, jumlahTim,
nextIdTim);
        else if (pilihan == 2) menuCariTimByNama(daftarTim, jumlahTim);
        else if (pilihan == 3) running = false;
    } while (running);
}

void menuAdmin(User* currentUser, Tim* daftarTim, int* jumlahTim, int*
nextIdTim) {
    int pilihan;
    bool running = true;
    do {
        clearScreen();
        tampilkanHeader("MENU ADMIN");
        cout << "User: " << currentUser->username << " | " <<
currentUser->namaLengkap << "\n\n";
        cout << "1. Tambah Pemain ke Tim\n";
        cout << "2. Lihat Semua Tim\n";
        cout << "3. Tambah Tim Baru\n";
        cout << "4. Hapus Tim dari Liga\n";
        cout << "5. Lihat Detail Tim dan Pemain\n";
        cout << "6. Cari Tim\n";
        cout << "7. Logout\n\n";
    } while (running);
}

```

```

        pilihan = Integer("Pilih menu (1-7): ", 1, 7);

        switch (pilihan) {
            case 1: tambahPemainMenu(daftarTim, jumlahTim, nextIdTim);
break;
            case 2: lihatSemuaTim(daftarTim, jumlahTim); break;
            case 3: tambahTimMenu(daftarTim, jumlahTim, nextIdTim);
break;
            case 4: hapusTimMenu(daftarTim, jumlahTim, nextIdTim);
break;
            case 5: lihatDetailTim(daftarTim, jumlahTim, nextIdTim);
break;
            case 6: menuSearching(daftarTim, jumlahTim, nextIdTim);
break;
            case 7: running = false; break;
        }
    } while (running);
}

void menuUser(User* currentUser, Tim* daftarTim, int* jumlahTim, int*
nextIdTim) {
    int pilihan;
    bool running = true;
    do {
        clearScreen();
        tampilkanHeader("MENU USER");
        cout << "User: " << currentUser->username << " | " <<
currentUser->namaLengkap << "\n\n";
        cout << "1. Lihat Semua Tim\n";
        cout << "2. Lihat Detail Tim dan Pemain\n";
        cout << "3. Cari Tim\n";
        cout << "4. Logout\n\n";

        pilihan = Integer("Pilih menu (1-4): ", 1, 4);

        switch (pilihan) {
            case 1: lihatSemuaTim(daftarTim, jumlahTim); break;
            case 2: lihatDetailTim(daftarTim, jumlahTim, nextIdTim);
break;
            case 3: menuSearching(daftarTim, jumlahTim, nextIdTim);
break;
            case 4: running = false; break;
        }
    } while (running);
}

void inisialisasiData(Tim* daftarTim, int* jumlahTim, int* nextIdTim) {
    User admin{"rando", "2509106093", "Admin", "Administrator"};
    databaseUsers.addUser(admin);

    User usr{"rando1", "093", "User", "User"};
    databaseUsers.addUser(usr);

    if (*jumlahTim == 0) {
        inisialisasiTim(&daftarTim[*jumlahTim], (*nextIdTim)++,

```

```

"Manchester City", "Etihad Stadium", 1880, 73);
    (*jumlahTim)++;
    inisialisasiTim(&daftarTim[*jumlahTim], (*nextIdTim)++,
"Arsenal", "Emirates Stadium", 1886, 71);
    (*jumlahTim)++;
    inisialisasiTim(&daftarTim[*jumlahTim], (*nextIdTim)++,
"Liverpool", "Anfield", 1892, 70);
    (*jumlahTim)++;
    }
}

void cleanupData(Tim* daftarTim, int* jumlahTim) {
    for (int i = 0; i < *jumlahTim; i++) {
        delete[] daftarTim[i].skuad;
    }
    *jumlahTim = 0;
}

int main() {
    Tim daftarTim[MAKS_TIM];
    int jumlahTim = 0;
    int nextIdTim = 1;

    for (int i = 0; i < MAKS_TIM; i++) daftarTim[i].skuad = nullptr;

    inisialisasiData(daftarTim, &jumlahTim, &nextIdTim);

    int pilihan;
    bool appRunning = true;

    while (appRunning) {
        clearScreen();
        tampilkanHeader("SISTEM MANAJEMEN LIGA SEPAK BOLA");
        cout << "1. Login\n";
        cout << "2. Registrasi\n";
        cout << "3. Keluar\n\n";

        try {
            pilihan = Integer("Pilih menu (1-3): ", 1, 3);

            switch (pilihan) {
                case 1: {
                    User currentUser;
                    loginMenu(&currentUser);
                    if (currentUser.role == "Admin") {
                        menuAdmin(&currentUser, daftarTim, &jumlahTim,
&nextIdTim);
                    } else if (currentUser.role == "User") {
                        menuUser(&currentUser, daftarTim, &jumlahTim,
&nextIdTim);
                    }
                    break;
                }
                case 2:
                    registrasiMenu();
            }
        }
    }
}

```

```

        break;
    case 3:
        cout << "\nTerima kasih! Sampai jumpa.\n";
        appRunning = false;
        break;
    }
} catch (...) {
    cout << "Terjadi kesalahan di menu utama.\n";
    clearInputBuffer();
}
}

cleanupData(daftarTim, &jumlahTim);
return 0;
}

```

4. Hasil Output

```
=====
      SISTEM MANAJEMEN LIGA PREMIER
=====
Selamat datang di Sistem Manajemen Liga Premier

1. Registrasi Akun Baru
2. Login
3. Keluar

Pilih menu (1-3):
```

Gambar 4.1 output menu awal

```
=====
      REGISTRASI AKUN BARU
=====
Masukkan Username: randooioi
Masukkan Password (minimal 6 karakter): randooioio
Masukkan Nama Lengkap: randoo

=====
      REGISTRASI BERHASIL!
=====
Username      : randooioi
Nama          : randoo
Role          : User

=====
Silakan login menggunakan akun Anda.

Tekan Enter untuk melanjutkan...█
```

Gambar 4.2 output registrasi akun sebagai user


```
=====
MENU LOGIN
=====
Username: rando123
Password: rando123
-----

LOGIN BERHASIL!
Selamat datang, rando!
Role: Admin
-----

Tekan Enter untuk melanjutkan...█
```

Gambar 4.3 output login menu berhasil

```
=====
LOGIN GAGAL
=====

Anda telah gagal login 3 kali.
Tekan Enter untuk keluar...
PS D:\praktikum-apl\post-test\post-test-2> █
```

Gambar 4.4 output ini ketika kita salah password hingga 3 kali

```
=====
      MENU ADMIN
=====
User: rando | Administrator

1. Tambah Pemain ke Tim
2. Lihat Semua Tim
3. Tambah Tim Baru
4. Hapus Tim dari Liga
5. Lihat Detail Tim dan Pemain
6. Cari Tim
7. Logout

Pilih menu (1-7): █
```

Gambar 4.5 output menu admin

```
=====
      TAMBAH PEMAIN KE TIM
=====
DAFTAR TIM:
-----
ID: 1 | Manchester City (Pemain: 0/11)
ID: 2 | Arsenal (Pemain: 0/11)
ID: 3 | Liverpool (Pemain: 0/11)
-----

Masukkan ID Tim: 2

Menambah pemain untuk tim: Arsenal
Jumlah pemain saat ini: 0/11

Nama Pemain: rando
Nomor Punggung: 10
Posisi: st
Jumlah Gol: 9
Jumlah Assist: 10

Pemain rando berhasil ditambahkan ke Arsenal!
Tekan Enter untuk kembali...█
```

Gambar 4.6 output untuk menambahkan pemain ke team

```
=====
          DAFTAR SEMUA TIM
=====
```

ID	Nama Tim	Stadion	Tahun Berdiri	Poin	Jumlah Pemain
1	Manchester City	Etihad Stadium	1880	73	0
2	Arsenal	Emirates Stadium	1886	71	1
3	Liverpool	Anfield	1892	70	0

```
=====
Tekan Enter untuk kembali...█
```

Gambar 4.7 output untuk melihat semua team dan poin

```
=====
          TAMBAH TIM BARU
=====
Nama Tim: barcelona
Stadion: segiri
Tahun Berdiri: 1945
Poin Awal: 89

Tim barcelona berhasil ditambahkan!
ID Tim: 4
Tekan Enter untuk kembali...█
```

Gambar 4.8 output untuk menambahkan team baru ke liga

```

=====
HAPUS TIM DARI LIGA
=====
DAFTAR TIM:
-----
ID    Nama Tim           Stadion              Poin    Jumlah Pemain
-----
1     Manchester City      Etihad Stadium      73      0
2     Arsenal               Emirates Stadium    71      1
3     Liverpool              Anfield             70      0
4     barcelona              segiri              89      0
=====

Masukkan ID Tim yang akan dihapus: 1

Anda akan menghapus tim:
Nama Tim      : Manchester City
Stadion       : Etihad Stadium
Jumlah Pemain: 0 pemain

Yakin ingin menghapus tim ini? (y/n): y

Tim Manchester City berhasil dihapus dari liga!

Tekan Enter untuk kembali...

```

Gambar 4.9 output untuk menghapus team

```

=====
DETAIL TIM
=====
ID Tim      : 2
Nama Tim    : Arsenal
Stadion     : Emirates Stadium
Tahun Berdiri : 1886
Poin        : 71
Jumlah Pemain : 1/11

DAFTAR PEMAIN:
-----
No    Nama Pemain           No Punggung    Posisi    Gol    Assist
-----
1     rando                   10             st        9      10
=====

Tekan Enter untuk kembali...

```

Gambar 5.0 output ingin melihat detail team

```
=====
MENU PENCARIAN TIM
=====
Pilih metode pencarian:

1. Cari Tim berdasarkan ID
2. Cari Tim berdasarkan Nama
3. Kembali

Pilih menu (1-3): █
```

Gambar 5.1 output menu pencarian tim metode Searching

```
=====
CARI TIM BERDASARKAN ID
=====
-----
ID Tim yang tersedia: 1, 2, 3
Masukkan ID Tim yang dicari: 2
Mencari tim dengan ID = 2 ...
-----
[DITEMUKAN] Tim dengan ID 2 berhasil ditemukan!

ID   Nama Tim           Stadion              Tahun    Poin    Pemain
-----
2    Arsenal             Emirates Stadium     1886     71      0
=====

Tekan Enter untuk kembali...█
```

Gambar 5.2 output melihat team berdasarkan id dengan metode Binary Search

```
=====
CARI TIM BERDASARKAN NAMA
=====
-----
Masukkan nama tim (atau sebagian nama): arsenal

Mencari tim dengan kata kunci "arsenal" ...
-----

[DITEMUKAN] 1 tim ditemukan:

ID   Nama Tim           Stadion              Tahun      Poin      Pemain
-----
2    Arsenal            Emirates Stadium    1886       71        0
=====

Tekan Enter untuk kembali...|
```

Gambar 5.3 output mencari team berdasarkan nama tim dengan metode linier search

```
=====
MENU USER
=====
User: rando1 | User

1. Lihat Semua Tim
2. Lihat Detail Tim dan Pemain
3. Cari Tim
4. Logout

Pilih menu (1-4): |
```

Gambar 5.4 output menu user

```
=====
DAFTAR SEMUA TIM
=====
ID   Nama Tim           Stadion              Tahun Berdiri  Poin      Jumlah Pemain
-----
2    Arsenal            Emirates Stadium    1886           71        1
3    Liverpool          Anfield            1892           70        0
4    barcelona          segiri             1945           89        0
=====

Tekan Enter untuk kembali...|
```

Gambar 5.5 output untuk melihat semua team dan poin


```
=====
      DETAIL TIM
=====
ID Tim      : 2
Nama Tim    : Arsenal
Stadion     : Emirates Stadium
Tahun Berdiri : 1886
Poin        : 71
Jumlah Pemain : 1/11

DAFTAR PEMAIN:
-----
No   Nama Pemain           No Punggung   Posisi      Gol   Assist
-----
1    rando                 10           st           9     10
=====

Tekan Enter untuk kembali...█
```

Gambar 5.6 output ingin melihat detail team

```
=====
      MENU PENCARIAN TIM
=====
Pilih metode pencarian:

1. Cari Tim berdasarkan ID
2. Cari Tim berdasarkan Nama
3. Kembali

Pilih menu (1-3): █
```

Gambar 5.7 output menu pencarian tim metode Searching

```

=====
CARI TIM BERDASARKAN ID
=====
-----

ID Tim yang tersedia: 1, 2, 3
Masukkan ID Tim yang dicari: 2
Mencari tim dengan ID = 2 ...
-----

[DITEMUKAN] Tim dengan ID 2 berhasil ditemukan!

```

ID	Nama Tim	Stadion	Tahun	Poin	Pemain
2	Arsenal	Emirates Stadium	1886	71	0

```

=====
Tekan Enter untuk kembali...

```

Gambar 5.8 output melihat team berdasarkan id dengan metode Binary Search

```

=====
CARI TIM BERDASARKAN NAMA
=====
-----

Masukkan nama tim (atau sebagian nama): arsenal
Mencari tim dengan kata kunci "arsenal" ...
-----

[DITEMUKAN] 1 tim ditemukan:

```

ID	Nama Tim	Stadion	Tahun	Poin	Pemain
2	Arsenal	Emirates Stadium	1886	71	0

```

=====
Tekan Enter untuk kembali...

```

Gambar 5.9 output mencari team berdasarkan nama tim dengan metode linier search


```
=====
SISTEM MANAJEMEN LIGA PREMIER
=====
Selamat datang di Sistem Manajemen Liga Premier

1. Registrasi Akun Baru
2. Login
3. Keluar

Pilih menu (1-3): 3

Terima kasih telah menggunakan program ini!
PS D:\praktikum-apl\post-test\post-test-2> █
```

Gambar 6.0 output untuk keluar dari program

5. Langkah-langkah GIT

5.1 GIT Add

```
PS D:\praktikum-apl\post-test\post-test-7> git add .
```

Gambar 5.1 git add itu dipakai untuk menandai file yang berubah supaya siap disimpan ke Git dengan git commit.

5.2 GIT Commit

```
PS D:\praktikum-apl\post-test\post-test-7> git commit -m "posttest7"
[main 3acc352] posttest7
2 files changed, 837 insertions(+)
create mode 100644 post-test/post-test-7/2509106093-israndomirhjah-pt-7.cpp
create mode 100644 post-test/post-test-7/2509106093-israndomirhjah-pt-7.exe
```

Gambar 5.2 git commit itu kayak **menyimpan catatan resmi** dari perubahan yang sudah kamu siapkan dengan git add.

5.3 GIT Push

```
PS D:\praktikum-apl\post-test\post-test-7> git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 691.54 KiB | 4.29 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object
To https://github.com/israndomirhjah/praktikum-apl.git
819bbf3..3acc352  main -> main
```

Gambar 5.3 git push itu dipakai untuk **mengirim commit yang ada di komputer lokal ke server Git (misalnya GitHub, GitLab, atau Bitbucket)**. Jadi perubahan yang sudah kamu simpan dengan git commit akan dikirim ke repository online supaya bisa dilihat atau dipakai orang lain.